



AUTHOR / DEVELOPER

Mayank Bisht

B.Tech in Computer Science & Engineering

PROJECT DOMAIN & TECH STACK

Healthcare AI, NLP, Computer Vision

Python, Streamlit, LLaMA 3, Tesseract OCR

⚠️ Ethical AI & Legal Disclaimer: MedSafe AI is built strictly as an educational tool and preventive awareness workspace. It does not provide professional medical diagnosis, treatment, or clinical advice. All clinical decisions must be guided by certified medical practitioners.

1. Executive Summary

MedSafe AI is a local-first healthcare safety application designed to bridge the gap between complex medical documentation and patient health literacy. By integrating fuzzy string matching algorithms, Optical Character Recognition (OCR), and Large Language Models (LLM) running 100% locally via Ollama, MedSafe AI provides an intuitive interface for evaluating drug interactions, parsing handwritten/printed prescriptions, and assessing symptom risk levels without exposing sensitive patient health information to third-party cloud APIs.

2. Problem Statement & Objectives

Modern healthcare systems face critical challenges in patient safety and prescription readability:

- **Illegible Prescriptions & Medical Jargon:** Medical prescriptions often contain specialized medical abbreviations, dense terms, or low-quality printing that lead to patient misinterpretation.
- **Adverse Drug Interactions (ADIs):** Patients taking multiple medications over-the-counter or across multiple doctors face unmonitored risk from severe drug interactions.
- **Symptom Missevaluation:** Patients frequently struggle to differentiate between symptoms requiring immediate emergency care vs. routine home observation.

Core Project Objectives

1. **Fuzzy Medicine Matching:** Map misspelled user inputs to standardized canonical drug registries using string similarity metrics.
2. **Local AI OCR Extraction:** Extract unstructured text from prescription images using Tesseract OCR and structure it into standardized JSON payloads using LLaMA 3.
3. **Deterministic Triage & Risk Evaluation:** Implement a hybrid rule-based emergency engine to detect clinical red flags alongside LLaMA 3 educational summaries.
4. **Privacy-Preserving Local Compute:** Execute all AI models offline via Ollama to ensure complete data security and patient privacy.

3. Key Modules & Functional Scenarios

Scenario 1: Medicine Interaction Analysis

Users enter comma-separated medication names (e.g., `Aspirine, Warfren`). The system utilizes **RapidFuzz** string matching with a 60% confidence threshold to match typos to official medicine entities. Recognized pairs are cross-referenced against a local interaction matrix. For flagged pairs, LLaMA 3 generates plain-language, non-technical explanations highlighting side effects and safety warnings.

Scenario 2: Prescription OCR & AI Structuring

Users upload an image of a medical prescription. **PyTesseract** converts visual pixels into raw text. The extracted raw string is fed into **LLaMA 3** with structured JSON formatting prompts. Regex parsers clean the output into a structured table containing `Name`, `Dosage`, and `Frequency` columns.

Scenario 3: Symptom Guidance & Emergency Risk Engine

Users report symptoms along with demographics (Age, Gender). A deterministic **Rule Engine** scans for critical red-flag terms (e.g., *chest pain*, *shortness of breath*) to calculate a 1–10 Emergency Risk Score. LLaMA 3 supplements the score with non-definitive educational possibilities, comfort measures, and emergency room warning indicators.

4. System Architecture & Tech Stack

```
[ User Interface: Streamlit Dashboard ]
├── Module 1: RapidFuzz Matching → Local JSON Database → Ollama (LLaMA 3)
├── Module 2: Tesseract OCR Engine → Regex JSON Parser → Ollama (LLaMA 3)
└── Module 3: Rule-Based Red Flag Engine → Ollama (LLaMA 3)
```

Component	Technology Used	Role in System
Frontend Framework	Streamlit	Interactive multi-tab web application shell
Generative AI	LLaMA 3 (via Ollama)	Local, offline LLM for structured JSON extraction & plain-language guidance
OCR Engine	Tesseract OCR & PyTesseract	Optical Character Recognition for prescription images
String Matching	RapidFuzz	Weighted ratio fuzzy string algorithm for drug name typo resolution

Component	Technology Used	Role in System
Data Persistence	JSON Data Files	Local database for medicine dictionary and drug interaction matrices

5. Project Structure

The codebase is organized into modular Python files adhering to clean architecture principles:

- `app.py` — Streamlit application script containing layout tabs and user UI components.
- `matcher.py` — Fuzzy string matching logic and interaction evaluation functions.
- `ocr_engine.py` — Image preprocessing and PyTesseract OCR extraction pipeline.
- `llm_engine.py` — Ollama / LLaMA 3 prompt templates, JSON regex extraction, and risk assessment logic.
- `data/medicines.json` — Local database storage containing standardized medicine names and interaction pairs.

6. Conclusion & Future Roadmap

The **MedSafe AI** project successfully demonstrates the integration of modern generative AI and computer vision into an offline, privacy-first healthcare safety assistant. By combining deterministic rules for emergency scoring with flexible AI reasoning for language generation, the application minimizes hallucination risks while offering maximum educational value.

Future Enhancements:

- Expansion of the local drug interaction database using RxNorm open clinical datasets.
- Integration with lightweight specialized medical models (e.g., BioMistral) for higher clinical precision.
- Support for multi-language prescription translation and voice-guided symptom assessment.